

# Guía Rápida de Numpy: Manipulación Eficiente y Procesamiento de Imágenes

## Bootcamp Data Science con BIT – Clase 6

---

### **Objetivo**

Tener a la mano los comandos y trucos más útiles de Numpy para operar con arrays, analizar datos numéricos y manipular imágenes.

---

### **¿Por qué usar Numpy?**

- Mucho más rápido y eficiente que listas o bucles de Python.
- Ideal para álgebra lineal, procesamiento de imágenes y cálculo científico.
- Base de casi todas las librerías modernas de Machine Learning (scikit-learn, TensorFlow, PyTorch, etc.).

### **Diferencia rápida:**

| <b>Pandas</b>              | <b>Numpy</b>                                  |
|----------------------------|-----------------------------------------------|
| Tablas (DataFrame)         | Arrays/matrices (ndarray)                     |
| Columnas/filas con nombre  | Índices numéricos                             |
| Ideal para datos tabulares | Ideal para números y operaciones vectorizadas |

---

### **1. Creación y exploración de arrays**

```
import numpy as np
```

```
np.array([1,2,3])
```

```
np.zeros((2,3))
```

```
np.ones((4,))
```

```
np.arange(10)
```

```
np.linspace(0, 1, 5)
```

- Forma (shape): arr.shape
  - Tamaño: arr.size
  - Tipo: arr.dtype
- 

## 2. Indexación y slicing

```
A = np.arange(9).reshape(3,3)
```

```
A[0, 2] # Fila 0, columna 2
```

```
A[:, 1] # Todas las filas, col 1
```

```
A[1:, :] # Desde fila 1 en adelante
```

---

## 3. Operaciones rápidas

```
arr + 10 # Suma 10 a todo el array
```

```
arr * 2 # Multiplica todo x2
```

```
arr > 5 # Array booleano
```

```
arr[arr < 5] = 0 # Asignar por condición
```

```
np.mean(arr)
```

```
np.std(arr)
```

```
np.max(arr, axis=1) # Máximo por fila
```

---

## 4. Filtrado y broadcasting

```
x = np.random.randint(0, 100, 10)
```

```
x[x > 50] # Elementos mayores a 50
```

```
x[x < x.mean()] = 0
```

---

## 🖼 5. Procesamiento básico de imágenes

- Leer imagen:

```
from PIL import Image  
  
img = Image.open('imagen.jpg')  
  
arr = np.array(img)  
  
arr.shape
```

- Mostrar con matplotlib:

```
import matplotlib.pyplot as plt  
  
plt.imshow(arr)  
  
plt.axis('off')  
  
plt.show()
```

---

## 🔗 Recursos útiles

- [Documentación oficial numpy](#)
- [Numpy Quickstart Tutorial](#)
- [Cheat Sheet numpy](#)